HW8

Zhen Wu

Prob. 3

To determine the minimizer of the non-smooth objective function $f(x, y) = |xy - 4| + (x - 7)^2 + (y - 3)^2$, we employ the Nelder-Mead simplex algorithm. This method is particularly suitable for functions where derivatives may be undefined or difficult to compute, as it relies solely on function evaluations.

```r
# Define the objective function
f <- function(p) {
  x <- p[1]
  y <- p[2]
  return(abs(x * y - 4) + (x - 7)^2 + (y - 3)^2)
}

# Initial guess
initial_guess <- c(0, 0)

# Execute the Nelder-Mead optimization using optim()
result <- optim(par = initial_guess, fn = f, method = "Nelder-Mead")

# Output the results
cat("The minimizer (x, y) is:\n")
```

```
## The minimizer (x, y) is:
```

```r
print(result$par)
```

```
## [1] 6.7906897 0.5890424
```

```r
cat("\nThe minimum value f(x, y) is:", result$value, "\n")
```

```
##
## The minimum value f(x, y) is: 5.856531
```

```r
cat("Total function evaluations:", result$counts[1], "\n")
```

```
## Total function evaluations: 149
```

Prob. 4

Linear Convergence

Given the sequence $x_{n+1} = f(x_n)$ and the fixed point $x^* = f(x^*)$, we analyze the error $\epsilon_n = x_n - x^*$. By applying the Mean Value Theorem to $f(x)$, we have:

$$x_{n+1} - x^* = f(x_n) - f(x^*) = f'(\xi_n)(x_n - x^*)$$

where ξ_n lies between x_n and x^*. Taking the absolute value:

$$|\epsilon_{n+1}| = |f'(\xi_n)||\epsilon_n|$$

As $n \to \infty$, $x_n \to x^*$ and $\xi_n \to x^*$. Since f' is continuous, we take the limit:

$$\lim_{n\to\infty} \frac{|\epsilon_{n+1}|}{|\epsilon_n|} = \lim_{n\to\infty} |f'(\xi_n)| = |f'(x^*)|$$

By definition, if $0 < |f'(x^*)| \leq k < 1$, the sequence converges linearly to x^* with a convergence rate $c = |f'(x^*)|$.

Superconvergence (Quadratic Convergence)

If $f'(x^*) = 0$, we perform a Taylor expansion of $f(x_n)$ around x^*:

$$x_{n+1} = f(x^*) + f'(x^*)(x_n - x^*) + \frac{f''(\xi_n)}{2}(x_n - x^*)^2$$

Since $f(x^*) = x^*$ and $f'(x^*) = 0$, this simplifies to:

$$x_{n+1} - x^* = \frac{f''(\xi_n)}{2}(x_n - x^*)^2 \implies \epsilon_{n+1} = \frac{f''(\xi_n)}{2}\epsilon_n^2$$

Assuming $f''(x)$ is continuous and $f''(x^*) \neq 0$, the ratio of errors becomes:

$$\lim_{n\to\infty} \frac{|\epsilon_{n+1}|}{|\epsilon_n|^2} = \frac{|f''(x^*)|}{2}$$

Thus, the condition for at least quadratic convergence is $f'(x^*) = 0$ and $f''(x^*)$ being bounded. In the dynamical systems literature, this state where the map is "flat" at the fixed point is referred to as superconvergence.

Prob. 5

To prove the convergence of the scheme $x^{(n+1)} = G(x^{(n)})$ with $G(x) = \frac{xg'(x)-g(x)}{g'(x)-1}$, we evaluate $G'(x^*)$. Using the quotient rule:

$$G'(x) = \frac{(g'(x) + xg''(x) - g'(x))(g'(x) - 1) - (xg'(x) - g(x))g''(x)}{(g'(x) - 1)^2}$$

$$G'(x) = \frac{xg''(x)(g'(x) - 1) - (xg'(x) - g(x))g''(x)}{(g'(x) - 1)^2}$$

At the fixed point x^*, we have $g(x^*) = x^*$. Substituting this into the numerator:

$$x^*g''(x^*)(g'(x^*) - 1) - (x^*g'(x^*) - x^*)g''(x^*) = x^*g''(x^*)(g'(x^*) - 1) - x^*(g'(x^*) - 1)g''(x^*) = 0$$

Since $g'(x^*) \neq 1$, the denominator is non-zero, so $G'(x^*) = 0$. By the logic established in Prob. 4, if $G'(x^*) = 0$, the sequence converges to x^* given an initial guess $x^{(0)}$ sufficiently close to x^*.

To find the order of convergence, we check $G''(x^*)$. If $G''(x^*) \neq 0$, the convergence is quadratic. This iteration is essentially a transformation of the fixed-point problem $g(x) = x$ into $g(x) - x = 0$, where $G(x)$ represents the Newton-Raphson operator applied to the function $h(x) = g(x) - x$. Since Newton's method typically exhibits quadratic convergence, we conclude the order is $\alpha = 2$.

Prob. 6

(1) Neumann Series

If $\rho(B) < 1$, the eigenvalues of B satisfy $|\lambda_i| < 1$. The eigenvalues of $I - B$ are $1 - \lambda_i \neq 0$, which implies $I - B$ is non-singular and $(I - B)^{-1}$ exists. The identity $(I - B)(I + B + B^2 + \cdots + B^k) = I - B^{k+1}$ holds. Since $\rho(B) < 1 \implies \lim_{k\to\infty} B^k = 0$, we have:

$$(I - B)\sum_{k=0}^{\infty} B^k = I \implies (I - B)^{-1} = \sum_{k=0}^{\infty} B^k$$

(2) Convergence and Bounds

The iteration $x^{(t+1)} = Bx^{(t)} + d$ has a unique solution $x^* = (I - B)^{-1}d$. Subtracting $x^* = Bx^* + d$ from the iteration formula:

$$x^{(t+1)} - x^* = B(x^{(t)} - x^*) \implies e^{(t+1)} = Be^{(t)}$$

Applying this recursively, $e^{(t)} = B^t e^{(0)}$. Taking the norm:

$$\|x^{(t)} - x^*\| \leq \|B^t\| \|x^{(0)} - x^*\| \leq \|B\|^t \|x^{(0)} - x^*\|$$

For the second bound, note $x^{(1)} - x^{(0)} = (x^{(1)} - x^*) - (x^{(0)} - x^*) = (B - I)(x^{(0)} - x^*)$. Thus $\|x^{(0)} - x^*\| \leq \|(I - B)^{-1}\| \|x^{(1)} - x^{(0)}\| \leq \frac{1}{1-\|B\|} \|x^{(1)} - x^{(0)}\|$. Substituting this into the first bound:

$$\|x^{(t)} - x^*\| \leq \frac{\|B\|^t}{1 - \|B\|} \|x^{(1)} - x^{(0)}\|$$

(3) Diagonal Dominance

If A is strictly diagonally dominant, then for the Jacobi method, $B_J = -D^{-1}(L + U)$. The entries are $b_{ij} = -a_{ij}/a_{ii}$. The infinity norm is $\|B_J\|_\infty = \max_i \sum_{j \neq i} |a_{ij}/a_{ii}|$. Since A is strictly diagonally dominant, $\sum_{j \neq i} |a_{ij}| < |a_{ii}|$, so $\|B_J\|_\infty < 1$. Since $\rho(B) \leq \|B\|$ for any induced norm, $\rho(B_J) < 1$, ensuring convergence. Similar logic applies to Gauss-Seidel using the Stein-Rosenberg Theorem.

Prob. 7

We first implement the Gauss-Seidel method and demonstrate its divergence on the original system.

```r
gauss_seidel <- function(A, b, x0, tol = 1e-7, max_iter = 100) {
  n <- length(b)
  x <- x0
  for (k in 1:max_iter) {
    x_old <- x
    for (i in 1:n) {
      sum_prev <- if (i > 1) sum(A[i, 1:(i-1)] * x[1:(i-1)]) else 0
      sum_next <- if (i < n) sum(A[i, (i+1):n] * x_old[(i+1):n]) else 0
      x[i] <- (b[i] - sum_prev - sum_next) / A[i, i]
    }
    if (sqrt(sum((x - x_old)^2)) < tol) return(list(x = x, iter = k, converged = TRUE))
    if (any(abs(x) > 1e10)) return(list(x = x, iter = k, converged = FALSE))
  }
  return(list(x = x, iter = max_iter, converged = FALSE))
}

# Original System
A_orig <- matrix(c(-1, 1, -3, 2, -1, 0, 1, -3, 1), nrow = 3, byrow = TRUE)
b_orig <- c(-6, 2, -2)
res_div <- gauss_seidel(A_orig, b_orig, c(0, 0, 0))
cat("Gauss-Seidel on original system converged:", res_div$converged, "\n")
```

```
## Gauss-Seidel on original system converged: FALSE
```

```r
# Reordered System (to be strictly diagonally dominant)
# Row 2 becomes Row 1: 2x1 - x2 = 2
# Row 3 becomes Row 2: x1 - 3x2 + x3 = -2
# Row 1 becomes Row 3: -x1 + x2 - 3x3 = -6
A_new <- matrix(c(2, -1, 0, 1, -3, 1, -1, 1, -3), nrow = 3, byrow = TRUE)
b_new <- c(2, -2, -6)
```

```r
res_conv <- gauss_seidel(A_new, b_new, c(0, 0, 0))

cat("Solution to two significant digits:\n")
```

```
## Solution to two significant digits:
```

```r
print(format(res_conv$x, digits = 2, nsmall = 1))
```

```
## [1] "2.0" "2.0" "2.0"
```

Prob. 8

We compare Gauss-Seidel and Successive Over-Relaxation (SOR) with $\omega = 1.25$.

```r
sor_method <- function(A, b, x0, omega, tol = 1e-9, max_iter = 1000) {
  n <- length(b)
  x <- x0
  for (k in 1:max_iter) {
    x_old <- x
    for (i in 1:n) {
      sum_prev <- if (i > 1) sum(A[i, 1:(i-1)] * x[1:(i-1)]) else 0
      sum_next <- if (i < n) sum(A[i, (i+1):n] * x_old[(i+1):n]) else 0
      x_gs <- (b[i] - sum_prev - sum_next) / A[i, i]
      x[i] <- (1 - omega) * x_old[i] + omega * x_gs
    }
    if (max(abs(x - x_old)) < tol) return(list(x = x, iter = k))
  }
  return(list(x = x, iter = max_iter))
}

A <- matrix(c(4, 3, 0, 3, 4, -1, 0, -1, 4), nrow = 3, byrow = TRUE)
b <- c(24, 30, -24)
x0 <- c(1, 1, 1)

# Solve using Gauss-Seidel (SOR with omega = 1)
res_gs <- sor_method(A, b, x0, omega = 1, tol = 1e-10)
# Solve using SOR with omega = 1.25
res_sor <- sor_method(A, b, x0, omega = 1.25, tol = 1e-10)

cat("Gauss-Seidel iterations:", res_gs$iter, "\n")
```

```
## Gauss-Seidel iterations: 46
```

```r
cat("SOR iterations (omega = 1.25):", res_sor$iter, "\n")
```

```
## SOR iterations (omega = 1.25): 20
```

```r
# Format output to 7 decimal places
cat("\nIterates accurate to 7 decimal places:\n")
```

```
##
## Iterates accurate to 7 decimal places:
```

```r
df <- data.frame(GS = res_gs$x, SOR = res_sor$x)
rownames(df) <- c("x1", "x2", "x3")
print(format(df, nsmall = 7))
```

```
##            GS        SOR
```

```
## x1  3.0000000  3.0000000
## x2  4.0000000  4.0000000
## x3 -5.0000000 -5.0000000
```